
8 *Arrays*

- What are Arrays
 - A Simple Program Using Array
- More on Arrays
 - Array Initialisation
 - Bounds Checking
 - Passing Array Elements to a Function
- Pointers and Arrays
 - Passing an Entire Array to a Function
 - The Real Thing
- Two Dimensional Arrays
 - Initialising a 2-Dimensional Array
 - Memory Map of a 2-Dimensional Array
 - Pointers and 2-Dimensional Arrays
 - Pointer to an Array
 - Passing 2-D Array to a Function
- Array of Pointers
- Three-Dimensional Array
- Summary
- Exercise

The C language provides a capability that enables the user to design a set of similar data types, called array. This chapter describes how arrays can be created and manipulated in C.

We should note that, in many C books and courses arrays and pointers are taught separately. I feel it is worthwhile to deal with these topics together. This is because pointers and arrays are so closely related that discussing arrays without discussing pointers would make the discussion incomplete and wanting. In fact all arrays make use of pointers internally. Hence it is all too relevant to study them together rather than as isolated topics.

What are Arrays

For understanding the arrays properly, let us consider the following program:

```
main()
{
    int x ;
    x = 5 ;
    x = 10 ;
    printf ( "\nx = %d", x ) ;
}
```

No doubt, this program will print the value of **x** as 10. Why so? Because when a value 10 is assigned to **x**, the earlier value of **x**, i.e. 5, is lost. Thus, ordinary variables (the ones which we have used so far) are capable of holding only one value at a time (as in the above example). However, there are situations in which we would want to store more than one value at a time in a single variable.

For example, suppose we wish to arrange the percentage marks obtained by 100 students in ascending order. In such a case we have two options to store these marks in memory:

- (a) Construct 100 variables to store percentage marks obtained by 100 different students, i.e. each variable containing one student's marks.
- (b) Construct one variable (called array or subscripted variable) capable of storing or holding all the hundred values.

Obviously, the second alternative is better. A simple reason for this is, it would be much easier to handle one variable than handling 100 different variables. Moreover, there are certain logics that cannot be dealt with, without the use of an array. Now a formal definition of an array—An array is a collective name given to a group of 'similar quantities'. These similar quantities could be percentage marks of 100 students, or salaries of 300 employees, or ages of 50 employees. What is important is that the quantities must be 'similar'. Each member in the group is referred to by its position in the group. For example, assume the following group of numbers, which represent percentage marks obtained by five students.

per = { 48, 88, 34, 23, 96 }

If we want to refer to the second number of the group, the usual notation used is per₂. Similarly, the fourth number of the group is referred as per₄. However, in C, the fourth number is referred as **per[3]**. This is because in C the counting of elements begins with 0 and not with 1. Thus, in this example **per[3]** refers to 23 and **per[4]** refers to 96. In general, the notation would be **per[i]**, where, **i** can take a value 0, 1, 2, 3, or 4, depending on the position of the element being referred. Here **per** is the subscripted variable (array), whereas **i** is its subscript.

Thus, an array is a collection of similar elements. These similar elements could be all **ints**, or all **floats**, or all **chars**, etc. Usually, the array of characters is called a 'string', whereas an array of **ints** or **floats** is called simply an array. Remember that all elements of

any given array must be of the same type. i.e. we cannot have an array of 10 numbers, of which 5 are **ints** and 5 are **floats**.

A Simple Program Using Array

Let us try to write a program to find average marks obtained by a class of 30 students in a test.

```
main()
{
    int avg, sum = 0 ;
    int i ;
    int marks[30] ; /* array declaration */

    for ( i = 0 ; i <= 29 ; i++ )
    {
        printf ( "\nEnter marks " ) ;
        scanf ( "%d", &marks[i] ) ; /* store data in array */
    }

    for ( i = 0 ; i <= 29 ; i++ )
        sum = sum + marks[i] ; /* read data from an array*/

    avg = sum / 30 ;
    printf ( "\nAverage marks = %d", avg ) ;
}
```

There is a lot of new material in this program, so let us take it apart slowly.

Array Declaration

To begin with, like other variables an array needs to be declared so that the compiler will know what kind of an array and how large an array we want. In our program we have done this with the statement:

```
int marks[30];
```

Here, **int** specifies the type of the variable, just as it does with ordinary variables and the word **marks** specifies the name of the variable. The **[30]** however is new. The number 30 tells how many elements of the type **int** will be in our array. This number is often called the ‘dimension’ of the array. The bracket ([]) tells the compiler that we are dealing with an array.

Accessing Elements of an Array

Once an array is declared, let us see how individual elements in the array can be referred. This is done with subscript, the number in the brackets following the array name. This number specifies the element’s position in the array. All the array elements are numbered, starting with 0. Thus, **marks[2]** is not the second element of the array, but the third. In our program we are using the variable **i** as a subscript to refer to various elements of the array. This variable can take different values and hence can refer to the different elements in the array in turn. This ability to use variables as subscripts is what makes arrays so useful.

Entering Data into an Array

Here is the section of code that places data into an array:

```
for ( i = 0 ; i <= 29 ; i++ )
{
    printf ( "\nEnter marks " );
    scanf ( "%d", &marks[i] );
}
```

The **for** loop causes the process of asking for and receiving a student’s marks from the user to be repeated 30 times. The first time through the loop, **i** has a value 0, so the **scanf()** function will cause the value typed to be stored in the array element **marks[0]**, the first element of the array. This process will be repeated until **i**

becomes 29. This is last time through the loop, which is a good thing, because there is no array element like **marks[30]**.

In **scanf()** function, we have used the “address of” operator (&) on the element **marks[i]** of the array, just as we have used it earlier on other variables (**&rate**, for example). In so doing, we are passing the address of this particular array element to the **scanf()** function, rather than its value; which is what **scanf()** requires.

Reading Data from an Array

The balance of the program reads the data back out of the array and uses it to calculate the average. The **for** loop is much the same, but now the body of the loop causes each student’s marks to be added to a running total stored in a variable called **sum**. When all the marks have been added up, the result is divided by 30, the number of students, to get the average.

```
for ( i = 0 ; i <= 29 ; i++ )
    sum = sum + marks[i] ;

avg = sum / 30 ;
printf ( "\nAverage marks = %d", avg ) ;
```

To fix our ideas, let us revise whatever we have learnt about arrays:

- (a) An array is a collection of similar elements.
- (b) The first element in the array is numbered 0, so the last element is 1 less than the size of the array.
- (c) An array is also known as a subscripted variable.
- (d) Before using an array its type and dimension must be declared.
- (e) However big an array its elements are always stored in contiguous memory locations. This is a very important point which we would discuss in more detail later on.

More on Arrays

Array is a very popular data type with C programmers. This is because of the convenience with which arrays lend themselves to programming. The features which make arrays so convenient to program would be discussed below, along with the possible pitfalls in using them.

Array Initialisation

So far we have used arrays that did not have any values in them to begin with. We managed to store values in them during program execution. Let us now see how to initialize an array while declaring it. Following are a few examples that demonstrate this.

```
int num[6] = { 2, 4, 12, 5, 45, 5 };  
int n[] = { 2, 4, 12, 5, 45, 5 };  
float press[] = { 12.3, 34.2, -23.4, -11.3 };
```

Note the following points carefully:

- (a) Till the array elements are not given any specific values, they are supposed to contain garbage values.
- (b) If the array is initialised where it is declared, mentioning the dimension of the array is optional as in the 2nd example above.

Array Elements in Memory

Consider the following array declaration:

```
int arr[8];
```

What happens in memory when we make this declaration? 16 bytes get immediately reserved in memory, 2 bytes each for the 8 integers (under Windows/Linux the array would occupy 32 bytes

as each integer would occupy 4 bytes). And since the array is not being initialized, all eight values present in it would be garbage values. This so happens because the storage class of this array is assumed to be **auto**. If the storage class is declared to be **static** then all the array elements would have a default initial value as zero. Whatever be the initial values, all the array elements would always be present in contiguous memory locations. This arrangement of array elements in memory is shown in Figure 8.1.

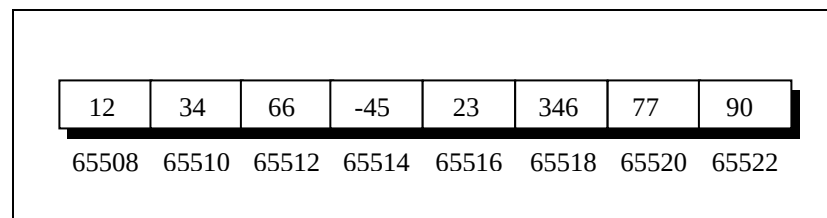


Figure 8.1

Bounds Checking

In C there is no check to see if the subscript used for an array exceeds the size of the array. Data entered with a subscript exceeding the array size will simply be placed in memory outside the array; probably on top of other data, or on the program itself. This will lead to unpredictable results, to say the least, and there will be no error message to warn you that you are going beyond the array size. In some cases the computer may just hang. Thus, the following program may turn out to be suicidal.

```
main()  
{  
    int num[40], i;  
  
    for ( i = 0 ; i <= 100 ; i++ )  
        num[i] = i;  
}
```

Thus, to see to it that we do not reach beyond the array size is entirely the programmer's botheration and not the compiler's.

Passing Array Elements to a Function

Array elements can be passed to a function by calling the function by value, or by reference. In the call by value we pass values of array elements to the function, whereas in the call by reference we pass addresses of array elements to the function. These two calls are illustrated below:

```
/* Demonstration of call by value */
main( )
{
    int i ;
    int marks[ ] = { 55, 65, 75, 56, 78, 78, 90 } ;

    for ( i = 0 ; i <= 6 ; i++ )
        display ( marks[i] ) ;
}

display ( int m )
{
    printf ( "%d ", m ) ;
}
```

And here's the output...

55 65 75 56 78 78 90

Here, we are passing an individual array element at a time to the function **display()** and getting it printed in the function **display()**. Note that since at a time only one element is being passed, this element is collected in an ordinary integer variable **m**, in the function **display()**.

And now the call by reference.

```
/* Demonstration of call by reference */
main()
{
    int i;
    int marks[] = { 55, 65, 75, 56, 78, 78, 90 };

    for ( i = 0 ; i <= 6 ; i++ )
        disp ( &marks[i] );
}

disp ( int *n )
{
    printf ( "%d ", *n );
}
```

And here's the output...

55 65 75 56 78 78 90

Here, we are passing addresses of individual array elements to the function **display()**. Hence, the variable in which this address is collected (**n**) is declared as a pointer variable. And since **n** contains the address of array element, to print out the array element we are using the 'value at address' operator (*****).

Read the following program carefully. The purpose of the function **disp()** is just to display the array elements on the screen. The program is only partly complete. You are required to write the function **show()** on your own. Try your hand at it.

```
main()
{
    int i;
    int marks[] = { 55, 65, 75, 56, 78, 78, 90 };

    for ( i = 0 ; i <= 6 ; i++ )
        disp ( &marks[i] );
}
```

```
}  
  
disp ( int *n )  
{  
    show ( &n );  
}
```

Pointers and Arrays

To be able to see what pointers have got to do with arrays, let us first learn some pointer arithmetic. Consider the following example:

```
main()  
{  
    int i = 3, *x;  
    float j = 1.5, *y;  
    char k = 'c', *z;  
  
    printf ( "\nValue of i = %d", i );  
    printf ( "\nValue of j = %f", j );  
    printf ( "\nValue of k = %c", k );  
    x = &i;  
    y = &j;  
    z = &k;  
    printf ( "\nOriginal address in x = %u", x );  
    printf ( "\nOriginal address in y = %u", y );  
    printf ( "\nOriginal address in z = %u", z );  
    x++;  
    y++;  
    z++;  
    printf ( "\nNew address in x = %u", x );  
    printf ( "\nNew address in y = %u", y );  
    printf ( "\nNew address in z = %u", z );  
}
```

Here is the output of the program.

```
Value of i = 3
Value of j = 1.500000
Value of k = c
Original address in x = 65524
Original address in y = 65520
Original address in z = 65519
New address in x = 65526
New address in y = 65524
New address in z = 65520
```

Observe the last three lines of the output. 65526 is original value in **x** plus 2, 65524 is original value in **y** plus 4, and 65520 is original value in **z** plus 1. This so happens because every time a pointer is incremented it points to the immediately next location of its type. That is why, when the integer pointer **x** is incremented, it points to an address two locations after the current location, since an **int** is always 2 bytes long (under Windows/Linux since **int** is 4 bytes long, new value of **x** would be 65528). Similarly, **y** points to an address 4 locations after the current location and **z** points 1 location after the current location. This is a very important result and can be effectively used while passing the entire array to a function.

The way a pointer can be incremented, it can be decremented as well, to point to earlier locations. Thus, the following operations can be performed on a pointer:

- (a) Addition of a number to a pointer. For example,

```
int i = 4, *j, *k ;
j = &i ;
j = j + 1 ;
j = j + 9 ;
k = j + 3 ;
```

- (b) Subtraction of a number from a pointer. For example,

```
int i = 4, *j, *k ;
j = &i ;
j = j - 2 ;
j = j - 5 ;
k = j - 6 ;
```

- (c) Subtraction of one pointer from another.

One pointer variable can be subtracted from another provided both variables point to elements of the same array. The resulting value indicates the number of bytes separating the corresponding array elements. This is illustrated in the following program.

```
main()
{
    int arr[] = { 10, 20, 30, 45, 67, 56, 74 } ;
    int *i, *j ;

    i = &arr[1] ;
    j = &arr[5] ;
    printf ( "%d %d", j - i, *j - *i ) ;
}
```

Here **i** and **j** have been declared as integer pointers holding addresses of first and fifth element of the array respectively.

Suppose the array begins at location 65502, then the elements **arr[1]** and **arr[5]** would be present at locations 65504 and 65512 respectively, since each integer in the array occupies two bytes in memory. The expression **j - i** would print a value 4 and not 8. This is because **j** and **i** are pointing to locations that are 4 integers apart. What would be the result of the expression ***j - *i**? 36, since ***j** and ***i** return the values present at addresses contained in the pointers **j** and **i**.

- (d) Comparison of two pointer variables

Pointer variables can be compared provided both variables point to objects of the same data type. Such comparisons can be useful when both pointer variables point to elements of the same array. The comparison can test for either equality or inequality. Moreover, a pointer variable can be compared with zero (usually expressed as NULL). The following program illustrates how the comparison is carried out.

```
main()  
{  
    int arr[] = { 10, 20, 36, 72, 45, 36 };  
    int *j, *k;  
  
    j = &arr[4];  
    k = (arr + 4);  
  
    if (j == k)  
        printf ("The two pointers point to the same location" );  
    else  
        printf ("The two pointers do not point to the same location" );  
}
```

A word of caution! Do not attempt the following operations on pointers... they would never work out.

- (a) Addition of two pointers
- (b) Multiplication of a pointer with a constant
- (c) Division of a pointer with a constant

Now we will try to correlate the following two facts, which we have learnt above:

- (a) Array elements are always stored in contiguous memory locations.
- (b) A pointer when incremented always points to an immediately next location of its type.

Suppose we have an array `num[ ] = { 24, 34, 12, 44, 56, 17 }`. The following figure shows how this array is located in memory.

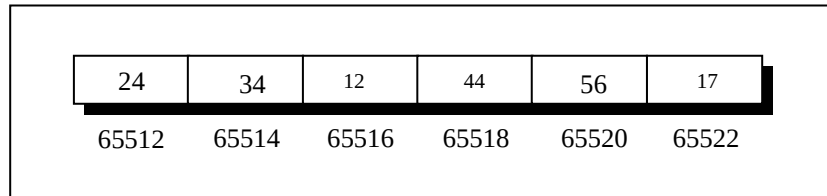


Figure 8.2

Here is a program that prints out the memory locations in which the elements of this array are stored.

```
main()  
{  
    int num[ ] = { 24, 34, 12, 44, 56, 17 };  
    int i;  
  
    for ( i = 0 ; i <= 5 ; i++ )  
    {  
        printf ( "\nelement no. %d ", i );  
        printf ( "address = %u", &num[i] );  
    }  
}
```

The output of this program would look like this:

```
element no. 0 address = 65512  
element no. 1 address = 65514  
element no. 2 address = 65516  
element no. 3 address = 65518  
element no. 4 address = 65520  
element no. 5 address = 65522
```

Note that the array elements are stored in contiguous memory locations, each element occupying two bytes, since it is an integer

array. When you run this program, you may get different addresses, but what is certain is that each subsequent address would be 2 bytes (4 bytes under Windows/Linux) greater than its immediate predecessor.

Our next two programs show ways in which we can access the elements of this array.

```
main()
{
    int num[] = { 24, 34, 12, 44, 56, 17 };
    int i;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\naddress = %u ", &num[i] );
        printf ( "element = %d", num[i] );
    }
}
```

The output of this program would be:

```
address = 65512 element = 24
address = 65514 element = 34
address = 65516 element = 12
address = 65518 element = 44
address = 65520 element = 56
address = 65522 element = 17
```

This method of accessing array elements by using subscripted variables is already known to us. This method has in fact been given here for easy comparison with the next method, which accesses the array elements using pointers.

```
main()
{
    int num[] = { 24, 34, 12, 44, 56, 17 } ;
```

```
int i, *j;

j = &num[0]; /* assign address of zeroth element */

for ( i = 0 ; i <= 5 ; i++ )
{
    printf ( "\naddress = %u ", j );
    printf ( "element = %d", *j );
    j++ ; /* increment pointer to point to next location */
}
```

The output of this program would be:

```
address = 65512 element = 24
address = 65514 element = 34
address = 65516 element = 12
address = 65518 element = 44
address = 65520 element = 56
address = 65522 element = 17
```

In this program, to begin with we have collected the base address of the array (address of the 0th element) in the variable **j** using the statement,

```
j = &num[0]; /* assigns address 65512 to j */
```

When we are inside the loop for the first time, **j** contains the address 65512, and the value at this address is 24. These are printed using the statements,

```
printf ( "\naddress = %u ", j );
printf ( "element = %d", *j );
```

On incrementing **j** it points to the next memory location of its type (that is location no. 65514). But location no. 65514 contains the second element of the array, therefore when the **printf()**

statements are executed for the second time they print out the second element of the array and its address (i.e. 34 and 65514)... and so on till the last element of the array has been printed.

Obviously, a question arises as to which of the above two methods should be used when? Accessing array elements by pointers is **always** faster than accessing them by subscripts. However, from the point of view of convenience in programming we should observe the following:

Array elements should be accessed using pointers if the elements are to be accessed in a fixed order, say from beginning to end, or from end to beginning, or every alternate element or any such definite logic.

Instead, it would be easier to access the elements using a subscript if there is no fixed logic in accessing the elements. However, in this case also, accessing the elements by pointers would work faster than subscripts.

Passing an Entire Array to a Function

In the previous section we saw two programs—one in which we passed individual elements of an array to a function, and another in which we passed addresses of individual elements to a function. Let us now see how to pass an entire array to a function rather than its individual elements. Consider the following example:

```
/* Demonstration of passing an entire array to a function */
main( )
{
    int num[ ] = { 24, 34, 12, 44, 56, 17 };
    display ( &num[0], 6 );
}

display ( int *j, int n )
{
```

```
int i;  
  
for ( i = 0 ; i <= n - 1 ; i++ )  
{  
    printf ( "\nelement = %d", *j ) ;  
    j++ ; /* increment pointer to point to next element */  
}  
}
```

Here, the **display()** function is used to print out the array elements. Note that the address of the zeroth element is being passed to the **display()** function. The **for** loop is same as the one used in the earlier program to access the array elements using pointers. Thus, just passing the address of the zeroth element of the array to a function is as good as passing the entire array to the function. It is also necessary to pass the total number of elements in the array, otherwise the **display()** function would not know when to terminate the **for** loop. Note that the address of the zeroth element (many a times called the base address) can also be passed by just passing the name of the array. Thus, the following two function calls are same:

```
display ( &num[0], 6 ) ;  
display ( num, 6 ) ;
```

The Real Thing

If you have grasped the concept of storage of array elements in memory and the arithmetic of pointers, here is some real food for thought. Once again consider the following array.

24	34	12	44	56	17
65512	65514	65516	65518	65520	65522

Figure 8.3

This is how we would declare the above array in C,

```
int num[] = { 24, 34, 12, 44, 56, 17 } ;
```

We also know that on mentioning the name of the array we get its base address. Thus, by saying ***num** we would be able to refer to the zeroth element of the array, that is, 24. One can easily see that ***num** and ***(num + 0)** both refer to 24.

Similarly, by saying ***(num + 1)** we can refer the first element of the array, that is, 34. In fact, this is what the C compiler does internally. When we say, **num[i]**, the C compiler internally converts it to ***(num + i)**. This means that all the following notations are same:

```
num[i]
*( num + i )
*( i + num )
i[num]
```

And here is a program to prove my point.

```
/* Accessing array elements in different ways */
main()
{
    int num[] = { 24, 34, 12, 44, 56, 17 } ;
    int i ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\naddress = %u ", &num[i] ) ;
        printf ( "element = %d %d ", num[i], *( num + i ) ) ;
        printf ( "%d %d", *( i + num ), i[num] ) ;
    }
}
```

The output of this program would be:

```
address = 65512 element = 24 24 24 24
address = 65514 element = 34 34 34 34
address = 65516 element = 12 12 12 12
address = 65518 element = 44 44 44 44
address = 65520 element = 56 56 56 56
address = 65522 element = 17 17 17 17
```

Two Dimensional Arrays

So far we have explored arrays with only one dimension. It is also possible for arrays to have two or more dimensions. The two-dimensional array is also called a matrix.

Here is a sample program that stores roll number and marks obtained by a student side by side in a matrix.

```
main( )
{
    int stud[4][2];
    int i, j;

    for ( i = 0 ; i <= 3 ; i++ )
    {
        printf ( "\n Enter roll no. and marks" );
        scanf ( "%d %d", &stud[i][0], &stud[i][1] );
    }

    for ( i = 0 ; i <= 3 ; i++ )
        printf ( "\n%d %d", stud[i][0], stud[i][1] );
}
```

There are two parts to the program—in the first part through a **for** loop we read in the values of roll no. and marks, whereas, in second part through another **for** loop we print out these values.

Look at the **scanf()** statement used in the first **for** loop:

```
scanf ( "%d %d", &stud[i][0], &stud[i][1] );
```

In **stud[i][0]** and **stud[i][1]** the first subscript of the variable **stud**, is row number which changes for every student. The second subscript tells which of the two columns are we talking about—the zeroth column which contains the roll no. or the first column which contains the marks. Remember the counting of rows and columns begin with zero. The complete array arrangement is shown below.

	col. no. 0	col. no. 1
row no. 0	1234	56
row no. 1	1212	33
row no. 2	1434	80
row no. 3	1312	78

Figure 8.4

Thus, 1234 is stored in **stud[0][0]**, 56 is stored in **stud[0][1]** and so on. The above arrangement highlights the fact that a two-dimensional array is nothing but a collection of a number of one-dimensional arrays placed one below the other.

In our sample program the array elements have been stored rowwise and accessed rowwise. However, you can access the array elements columnwise as well. Traditionally, the array elements are being stored and accessed rowwise; therefore we would also stick to the same strategy.

Initialising a 2-Dimensional Array

How do we initialize a two-dimensional array? As simple as this...

```
int stud[4][2] = {  
    { 1234, 56 },  
    { 1212, 33 },  
    { 1434, 80 },  
    { 1312, 78 }  
};
```

or even this would work...

```
int stud[4][2] = { 1234, 56, 1212, 33, 1434, 80, 1312, 78 } ;
```

of course with a corresponding loss in readability.

It is important to remember that while initializing a 2-D array it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional.

Thus the declarations,

```
int arr[2][3] = { 12, 34, 23, 45, 56, 45 } ;  
int arr[ ][3] = { 12, 34, 23, 45, 56, 45 } ;
```

are perfectly acceptable,

whereas,

```
int arr[2][ ] = { 12, 34, 23, 45, 56, 45 } ;  
int arr[ ][ ] = { 12, 34, 23, 45, 56, 45 } ;
```

would never work.

Memory Map of a 2-Dimensional Array

Let us reiterate the arrangement of array elements in a two-dimensional array of students, which contains roll nos. in one column and the marks in the other.

The array arrangement shown in Figure 8.4 is only conceptually true. This is because memory doesn't contain rows and columns. In memory whether it is a one-dimensional or a two-dimensional array the array elements are stored in one continuous chain. The arrangement of array elements of a two-dimensional array in memory is shown below:

s[0][0]	s[0][1]	s[1][0]	s[1][1]	s[2][0]	s[2][1]	s[3][0]	s[3][1]
1234	56	1212	33	1434	80	1312	78
65508	65510	65512	65514	65516	65518	65520	65522

Figure 8.5

We can easily refer to the marks obtained by the third student using the subscript notation as shown below:

```
printf ( "Marks of third student = %d", stud[2][1] );
```

Can we not refer the same element using pointer notation, the way we did in one-dimensional arrays? Answer is yes. Only the procedure is slightly difficult to understand. So, read on...

Pointers and 2-Dimensional Arrays

The C language embodies an unusual but powerful capability—it can treat parts of arrays as arrays. More specifically, each row of a two-dimensional array can be thought of as a one-dimensional array. This is a very important fact if we wish to access array elements of a two-dimensional array using pointers.

Thus, the declaration,

```
int s[5][2];
```

can be thought of as setting up an array of 5 elements, each of which is a one-dimensional array containing 2 integers. We refer to an element of a one-dimensional array using a single subscript. Similarly, if we can imagine **s** to be a one-dimensional array then we can refer to its zeroth element as **s[0]**, the next element as **s[1]** and so on. More specifically, **s[0]** gives the address of the zeroth one-dimensional array, **s[1]** gives the address of the first one-dimensional array and so on. This fact can be demonstrated by the following program.

```
/* Demo: 2-D array is an array of arrays */
main( )
{
    int s[4][2] = {
        { 1234, 56 },
        { 1212, 33 },
        { 1434, 80 },
        { 1312, 78 }
    };

    int i ;

    for ( i = 0 ; i <= 3 ; i++ )
        printf ( "\nAddress of %d th 1-D array = %u", i, s[i] ) ;
}
```

And here is the output...

```
Address of 0 th 1-D array = 65508
Address of 1 th 1-D array = 65512
Address of 2 th 1-D array = 65516
Address of 3 th 1-D array = 65520
```

Let's figure out how the program works. The compiler knows that **s** is an array containing 4 one-dimensional arrays, each containing 2 integers. Each one-dimensional array occupies 4 bytes (two bytes for each integer). These one-dimensional arrays are placed linearly (zeroth 1-D array followed by first 1-D array, etc.). Hence

each one-dimensional array starts 4 bytes further along than the last one, as can be seen in the memory map of the array shown below.

s[0][0]	s[0][1]	s[1][0]	s[1][1]	s[2][0]	s[2][1]	s[3][0]	s[3][1]
1234	56	1212	33	1434	80	1312	78
65508	65510	65512	65514	65516	65518	65520	65522

Figure 8.6

We know that the expressions **s[0]** and **s[1]** would yield the addresses of the zeroth and first one-dimensional array respectively. From Figure 8.6 these addresses turn out to be 65508 and 65512.

Now, we have been able to reach each one-dimensional array. What remains is to be able to refer to individual elements of a one-dimensional array. Suppose we want to refer to the element **s[2][1]** using pointers. We know (from the above program) that **s[2]** would give the address 65516, the address of the second one-dimensional array. Obviously (65516 + 1) would give the address 65518. Or (**s[2] + 1**) would give the address 65518. And the value at this address can be obtained by using the value at address operator, saying ***(s[2] + 1)**. But, we have already studied while learning one-dimensional arrays that **num[i]** is same as ***(num + i)**. Similarly, ***(s[2] + 1)** is same as, ***(*(s + 2) + 1)**. Thus, all the following expressions refer to the same element,

```
s[2][1]
*( s[2] + 1 )
*( *( s + 2 ) + 1 )
```

Using these concepts the following program prints out each element of a two-dimensional array using pointer notation.

```
/* Pointer notation to access 2-D array elements */
main( )
{
    int s[4][2] = {
        { 1234, 56 },
        { 1212, 33 },
        { 1434, 80 },
        { 1312, 78 }
    };

    int i, j;

    for ( i = 0 ; i <= 3 ; i++ )
    {
        printf ( "\n" );
        for ( j = 0 ; j <= 1 ; j++ )
            printf ( "%d ", *( s + i ) + j );
    }
}
```

And here is the output...

```
1234 56
1212 33
1434 80
1312 78
```

Pointer to an Array

If we can have a pointer to an integer, a pointer to a float, a pointer to a char, then can we not have a pointer to an array? We certainly can. The following program shows how to build and use it.

```
/* Usage of pointer to an array */
main()
{
    int s[5][2] = {
        { 1234, 56 },
        { 1212, 33 },
        { 1434, 80 },
        { 1312, 78 }
    };

    int (*p)[2];
    int i, j, *pint;

    for ( i = 0 ; i <= 3 ; i++ )
    {
        p = &s[i];
        pint = p;
        printf ( "\n" );
        for ( j = 0 ; j <= 1 ; j++ )
            printf ( "%d ", *( pint + j ) );
    }
}
```

And here is the output...

```
1234 56
1212 33
1434 80
1312 78
```

Here **p** is a pointer to an array of two integers. Note that the parentheses in the declaration of **p** are necessary. Absence of them would make **p** an array of 2 integer pointers. Array of pointers is covered in a later section in this chapter. In the outer **for** loop each time we store the address of a new one-dimensional array. Thus first time through this loop **p** would contain the address of the zeroth 1-D array. This address is then assigned to an integer pointer **pint**. Lastly, in the inner **for** loop using the pointer **pint** we

have printed the individual elements of the 1-D array to which **p** is pointing.

But why should we use a pointer to an array to print elements of a 2-D array. Is there any situation where we can appreciate its usage better? The entity pointer to an array is immensely useful when we need to pass a 2-D array to a function. This is discussed in the next section.

Passing 2-D Array to a Function

There are three ways in which we can pass a 2-D array to a function. These are illustrated in the following program.

/ Three ways of accessing a 2-D array */*

```
main( )
{
    int a[3][4] = {
        1, 2, 3, 4,
        5, 6, 7, 8,
        9, 0, 1, 6
    };

    clrscr( ) ;
    display ( a, 3, 4 ) ;
    show ( a, 3, 4 ) ;
    print ( a, 3, 4 ) ;
}
display ( int *q, int row, int col )
{
    int i, j ;

    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", * ( q + i * col + j ) ) ;
```

```
        printf ( "\n" );
    }
    printf ( "\n" );
}

show ( int ( *q )[4], int row, int col )
{
    int i, j ;
    int *p ;

    for ( i = 0 ; i < row ; i++ )
    {
        p = q + i ;
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", * ( p + j ) ) ;

        printf ( "\n" );
    }
    printf ( "\n" );
}

print ( int q[ ][4], int row, int col )
{
    int i, j ;

    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", q[i][j] ) ;
        printf ( "\n" );
    }
    printf ( "\n" );
}
```

And here is the output...

```
1 2 3 4
5 6 7 8
```

```
9 0 1 6
```

```
1 2 3 4
```

```
5 6 7 8
```

```
9 0 1 6
```

```
1 2 3 4
```

```
5 6 7 8
```

```
9 0 1 6
```

In the **display()** function we have collected the base address of the 2-D array being passed to it in an ordinary **int** pointer. Then through the two **for** loops using the expression $*(q + i * col + j)$ we have reached the appropriate element in the array. Suppose **i** is equal to 2 and **j** is equal to 3, then we wish to reach the element **a[2][3]**. Let us see whether the expression $*(q + i * col + j)$ does give this element or not. Refer Figure 8.7 to understand this.

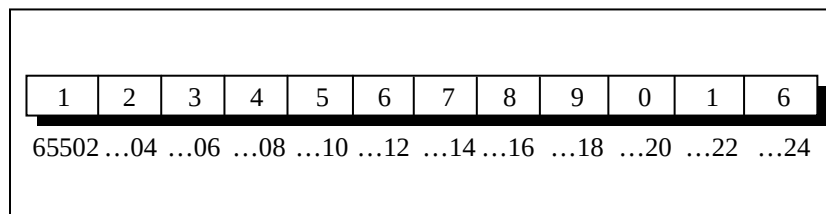


Figure 8.7

The expression $*(q + i * col + j)$ becomes $*(65502 + 2 * 4 + 3)$. This turns out to be $*(65502 + 11)$. Since 65502 is address of an integer, $*(65502 + 11)$ turns out to be $*(65524)$. Value at this address is 6. This is indeed same as **a[2][3]**. A more general formula for accessing each array element would be:

$*(base\ address + row\ no. * no.\ of\ columns + column\ no.)$

In the **show()** function we have defined **q** to be a pointer to an array of 4 integers through the declaration:


```
int ( *q )[4] ;
```

To begin with, **q** holds the base address of the zeroth 1-D array, i.e. 4001 (refer Figure 8.7). This address is then assigned to **p**, an **int** pointer, and then using this pointer all elements of the zeroth 1-D array are accessed. Next time through the loop when **i** takes a value 1, the expression **q + i** fetches the address of the first 1-D array. This is because, **q** is a pointer to zeroth 1-D array and adding 1 to it would give us the address of the next 1-D array. This address is once again assigned to **p**, and using it all elements of the next 1-D array are accessed.

In the third function **print()**, the declaration of **q** looks like this:

```
int q[ ][4] ;
```

This is same as **int (*q)[4]**, where **q** is pointer to an array of 4 integers. The only advantage is that we can now use the more familiar expression **q[i][j]** to access array elements. We could have used the same expression in **show()** as well.

Array of Pointers

The way there can be an array of **ints** or an array of **floats**, similarly there can be an array of pointers. Since a pointer variable always contains an address, an array of pointers would be nothing but a collection of addresses. The addresses present in the array of pointers can be addresses of isolated variables or addresses of array elements or any other addresses. All rules that apply to an ordinary array apply to the array of pointers as well. I think a program would clarify the concept.

```
main()  
{  
    int *arr[4] ; /* array of integer pointers */
```

```

int i = 31, j = 5, k = 19, l = 71, m ;

arr[0] = &i ;
arr[1] = &j ;
arr[2] = &k ;
arr[3] = &l ;

for ( m = 0 ; m <= 3 ; m++ )
    printf ( "%d ", * ( arr[m] ) ) ;
}

```

Figure 8.8 shows the contents and the arrangement of the array of pointers in memory. As you can observe, **arr** contains addresses of isolated **int** variables **i**, **j**, **k** and **l**. The **for** loop in the program picks up the addresses present in **arr** and prints the values present at these addresses.

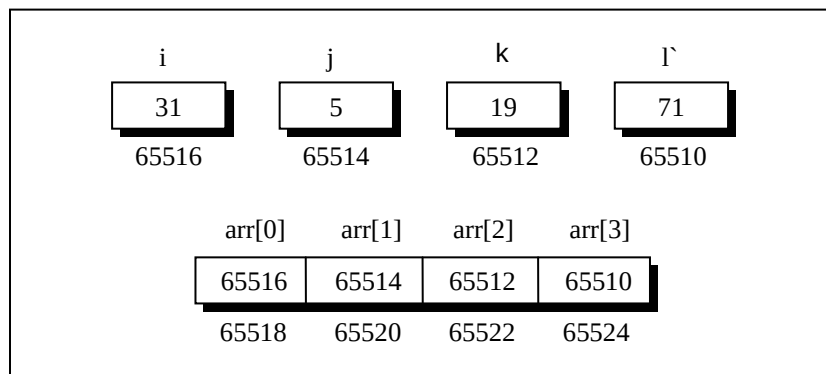


Figure 8.8

An array of pointers can even contain the addresses of other arrays. The following program would justify this.

```

main()
{
    static int a[] = { 0, 1, 2, 3, 4 };

```

```
int *p[] = { a, a + 1, a + 2, a + 3, a + 4 };

printf ( "\n%u %u %d", p, *p, * ( *p ) );
}
```

I would leave it for you to figure out the output of this program.

Three-Dimensional Array

We aren't going to show a programming example that uses a three-dimensional array. This is because, in practice, one rarely uses this array. However, an example of initializing a three-dimensional array will consolidate your understanding of subscripts:

```
int arr[3][4][2] = {
    {
        { 2, 4 },
        { 7, 8 },
        { 3, 4 },
        { 5, 6 }
    },
    {
        { 7, 6 },
        { 3, 4 },
        { 5, 3 },
        { 2, 3 }
    },
    {
        { 8, 9 },
        { 7, 2 },
        { 3, 4 },
        { 5, 1 },
    }
};
```

A three-dimensional array can be thought of as an array of arrays of arrays. The outer array has three elements, each of which is a

two-dimensional array of four one-dimensional arrays, each of which contains two integers. In other words, a one-dimensional array of two elements is constructed first. Then four such one-dimensional arrays are placed one below the other to give a two-dimensional array containing four rows. Then, three such two-dimensional arrays are placed one behind the other to yield a three-dimensional array containing three 2-dimensional arrays. In the array declaration note how the commas have been given. Figure 8.9 would possibly help you in visualising the situation better.

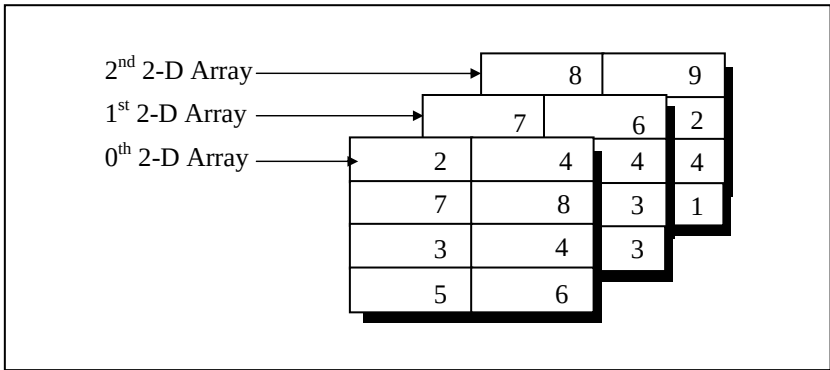


Figure 8.9

Again remember that the arrangement shown above is only conceptually true. In memory the same array elements are stored linearly as shown in Figure 8.10.

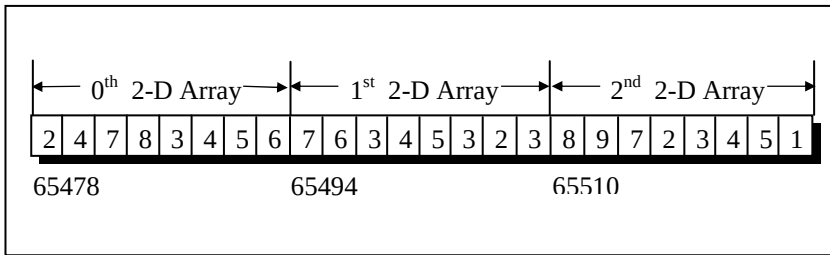


Figure 8.10

How would you refer to the array element 1 in the above array? The first subscript should be [2], since the element is in third two-dimensional array; the second subscript should be [3] since the element is in fourth row of the two-dimensional array; and the third subscript should be [1] since the element is in second position in the one-dimensional array. We can therefore say that the element 1 can be referred as **arr[2][3][1]**. It may be noted here that the counting of array elements even for a 3-D array begins with zero. Can we not refer to this element using pointer notation? Of course, yes. For example, the following two expressions refer to the same element in the 3-D array:

```
arr[2][3][1]
*( *( *( arr + 2 ) + 3 ) + 1 )
```

Summary

- (a) An array is similar to an ordinary variable except that it can store multiple elements of similar type.
- (b) Compiler doesn't perform bounds checking on an array.
- (c) The array variable acts as a pointer to the zeroth element of the array. In a 1-D array, zeroth element is a single value, whereas, in a 2-D array this element is a 1-D array.
- (d) On incrementing a pointer it points to the next location of its type.
- (e) Array elements are stored in contiguous memory locations and so they can be accessed using pointers.
- (f) Only limited arithmetic can be done on pointers.

Exercise

Simple arrays

[A] What would be the output of the following programs:

- (a) `main( )`

```
{
    int num[26], temp ;
    num[0] = 100 ;
    num[25] = 200 ;
    temp = num[25] ;
    num[25] = num[0] ;
    num[0] = temp ;
    printf ( "\n%d %d", num[0], num[25] ) ;
}

(b) main( )
{
    int array[26], i ;
    for ( i = 0 ; i <= 25 ; i++ )
    {
        array[i] = 'A' + i ;
        printf ( "\n%d %c", array[i], array[i] ) ;
    }
}

(c) main( )
{
    int sub[50], i ;
    for ( i = 0 ; i <= 48 ; i++ ) ;
    {
        sub[i] = i ;
        printf ( "\n%d", sub[i] ) ;
    }
}
```

[B] Point out the errors, if any, in the following program segments:

```
(a) /* mixed has some char and some int values */
    int char mixed[100] ;

    main( )
    {
        int a[10], i ;
```

```
        for ( i = 1 ; i <= 10 ; i++ )
        {
            scanf ( "%d", a[i] ) ;
            printf ( "%d", a[i] ) ;
        }
    }

(b)  main( )
    {
        int size ;
        scanf ( "%d", &size ) ;
        int arr[size] ;
        for ( i = 1 ; i <= size ; i++ )
        {
            scanf ( "%d", arr[i] ) ;
            printf ( "%d", arr[i] ) ;
        }
    }

(c)  main( )
    {
        int i, a = 2, b = 3 ;
        int arr[ 2 + 3 ] ;
        for ( i = 0 ; i < a+b ; i++ )
        {
            scanf ( "%d", &arr[i] ) ;
            printf ( "\n%d", arr[i] ) ;
        }
    }
```

[C] Answer the following:

(a) An array is a collection of

1. different data types scattered throughout memory
2. the same data type scattered throughout memory
3. the same data type placed next to each other in memory
4. different data types placed next to each other in memory

- (b) Are the following array declarations correct?

```
int a (25) ;  
int size = 10, b[size] ;  
int c = {0,1,2} ;
```

- (c) Which element of the array does this expression reference?

```
num[4]
```

- (d) What is the difference between the 5's in these two expressions? (Select the correct answer)

```
int num[5] ;  
num[5] = 11 ;
```

1. first is particular element, second is type
2. first is array size, second is particular element
3. first is particular element, second is array size
4. both specify array size

- (e) State whether the following statements are True or False:

1. The array **int num[26]** has twenty-six elements.
2. The expression **num[1]** designates the first element in the array
3. It is necessary to initialize the array at the time of declaration.
4. The expression **num[27]** designates the twenty-eighth element in the array.

[D] Attempt the following:

- (a) Twenty-five numbers are entered from the keyboard into an array. The number to be searched is entered through the keyboard by the user. Write a program to find if the number to be searched is present in the array and if it is present, display the number of times it appears in the array.

- (b) Twenty-five numbers are entered from the keyboard into an array. Write a program to find out how many of them are positive, how many are negative, how many are even and how many odd.
- (c) Implement the Selection Sort, Bubble Sort and Insertion sort algorithms on a set of 25 numbers. (Refer Figure 8.11 for the logic of the algorithms)
- Selection sort
 - Bubble Sort
 - Insertion Sort

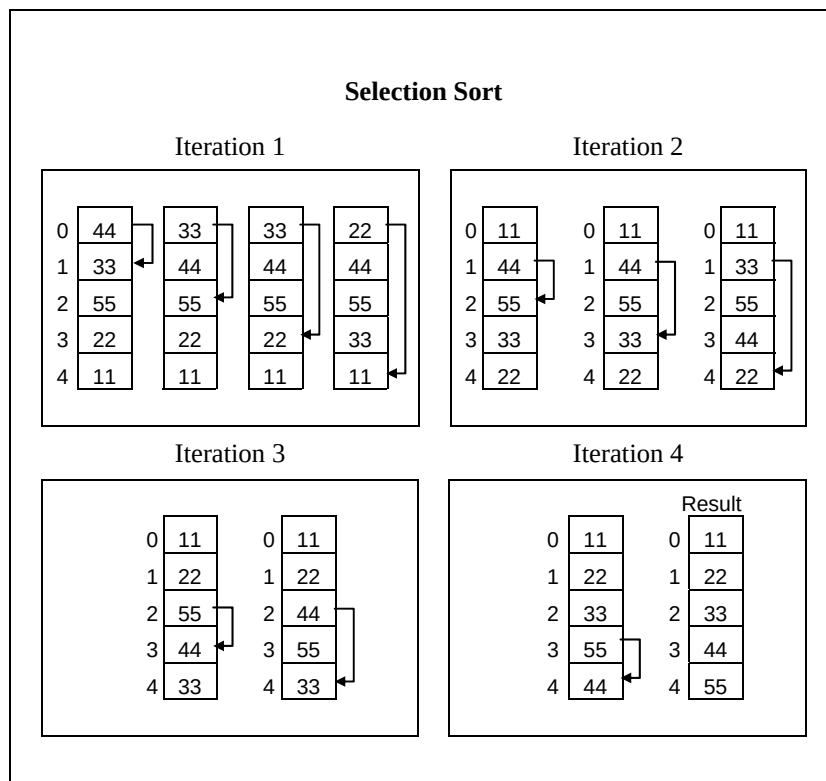


Figure 8.11 (a)

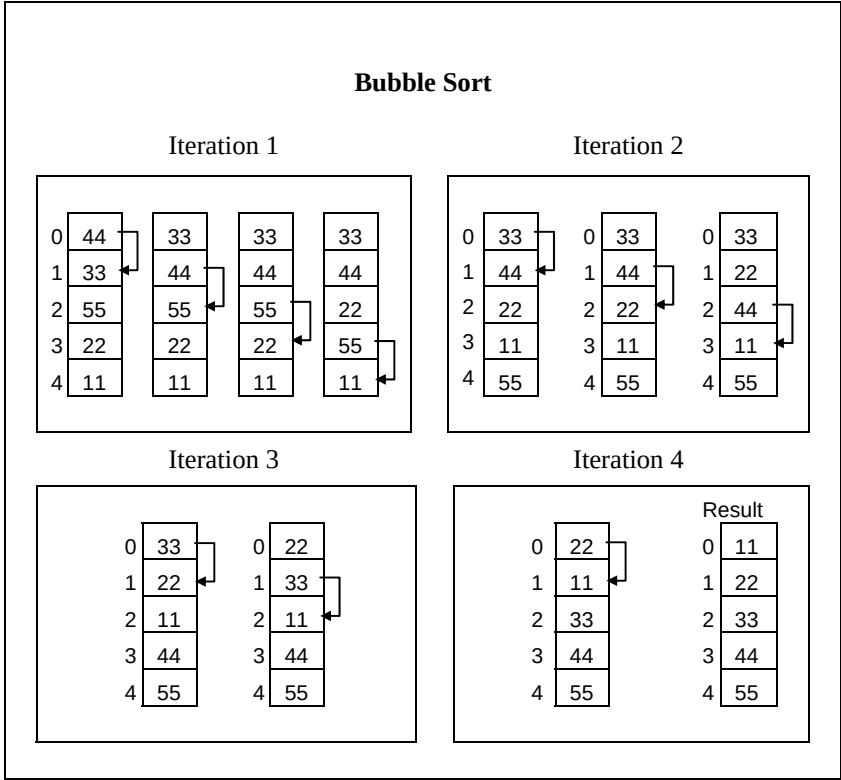


Figure 8.11 (b)

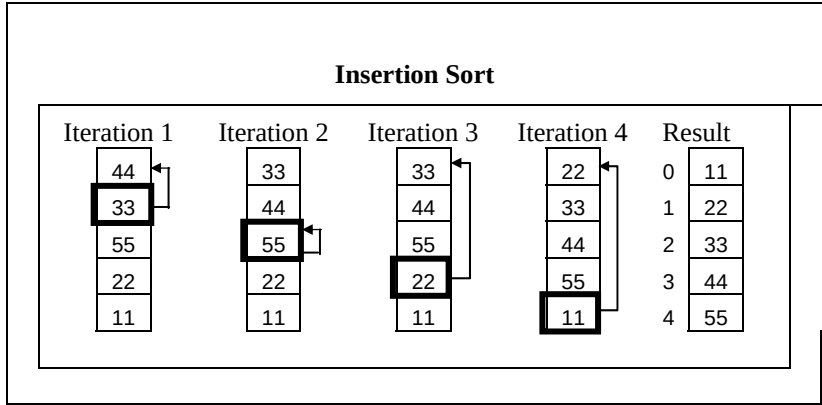


Figure 8.11 (c)

- (d) Implement the following procedure to generate prime numbers from 1 to 100 into a program. This procedure is called sieve of Eratosthenes.

- step 1 Fill an array **num[100]** with numbers from 1 to 100
- step 2 Starting with the second entry in the array, set all its multiples to zero.
- step 3 Proceed to the next non-zero element and set all its multiples to zero.
- step 4 Repeat step 3 till you have set up the multiples of all the non-zero elements to zero
- step 5 At the conclusion of step 4, all the non-zero entries left in the array would be prime numbers, so print out these numbers.

More on arrays, Arrays and pointers

[E] What would be the output of the following programs:

(a)

```
main()  
{  
    int b[] = { 10, 20, 30, 40, 50 };  
    int i;  
    for ( i = 0 ; i <= 4 ; i++)  
        printf ( "\n%d" *( b + i ) );  
}
```

(b)

```
main()  
{  
    int b[] = { 0, 20, 0, 40, 5 };  
    int i, *k;  
    k = b;  
    for ( i = 0 ; i <= 4 ; i++)  
    {  
        printf ( "\n%d" *k );  
    }
```

```
        k++ ;
    }
}

(c) main()
{
    int a[] = { 2, 4, 6, 8, 10 };
    int i ;
    change ( a, 5 );
    for ( i = 0 ; i <= 4 ; i++ )
        printf( "\n%d", a[i] );
}

change ( int *b, int n )
{
    int i ;
    for ( i = 0 ; i < n ; i++ )
        *( b + i ) = *( b + i ) + 5 ;
}

(d) main()
{
    int a[5], i, b = 16 ;
    for ( i = 0 ; i < 5 ; i++ )
        a[i] = 2 * i ;
    f ( a, b );
    for ( i = 0 ; i < 5 ; i++ )
        printf ( "\n%d", a[i] );
    printf( "\n%d", b );
}

f ( int *x, int y )
{
    int i ;
    for ( i = 0 ; i < 5 ; i++ )
        *( x + i ) += 2 ;
    y += 2 ;
}
```

```
(e)  main( )
    {
        static int a[5] ;
        int i ;
        for ( i = 0 ; i <= 4 ; i++ )
            printf ( "\n%d", a[i] ) ;
    }

(f)  main()
    {
        int a[5] = { 5, 1, 15, 20, 25 } ;
        int i, j, k = 1, m ;
        i = ++a[1] ;
        j = a[1]++ ;
        m = a[i++] ;
        printf ( "\n%d %d %d", i, j, m ) ;
    }
```

[F] Point out the errors, if any, in the following programs:

```
(a)  main( )
    {
        int array[6] = { 1, 2, 3, 4, 5, 6 } ;
        int i ;
        for ( i = 0 ; i <= 25 ; i++ )
            printf ( "\n%d", array[i] ) ;
    }

(b)  main()
    {
        int sub[50], i ;
        for ( i = 1 ; i <= 50 ; i++ )
        {
            sub[i] = i ;
            printf ( "\n%d" , sub[i] ) ;
        }
    }
```

- ```
(c) main()
 {
 int a[] = { 10, 20, 30, 40, 50 };
 int j;
 j = a; /* store the address of zeroth element */
 j = j + 3;
 printf ("\n%d" *j);
 }

(d) main()
 {
 float a[] = { 13.24, 1.5, 1.5, 5.4, 3.5 };
 float *j;
 j = a;
 j = j + 4;
 printf ("\n%d %d %d", j, *j, a[4]);
 }

(e) main()
 {
 float a[] = { 13.24, 1.5, 1.5, 5.4, 3.5 };
 float *j, *k;
 j = a;
 k = a + 4;
 j = j * 2;
 k = k / 2;
 printf ("\n%d %d", *j, *k);
 }

(f) main()
 {
 int max = 5;
 float arr[max];
 for (i = 0; i < max; i++)
 scanf ("%f", &arr[i]);
 }
```

[G] Answer the following:

- (a) What would happen if you try to put so many values into an array when you initialize it that the size of the array is exceeded?
  - 1. nothing
  - 2. possible system malfunction
  - 3. error message from the compiler
  - 4. other data may be overwritten
- (b) In an array **int arr[12]** the word **arr** represents the a\_\_\_\_\_ of the array
- (c) What would happen if you put too few elements in an array when you initialize it?
  - 1. nothing
  - 2. possible system malfunction
  - 3. error message from the compiler
  - 4. unused elements will be filled with 0's or garbage
- (d) What would happen if you assign a value to an element of an array whose subscript exceeds the size of the array?
  - 1. the element will be set to 0
  - 2. nothing, it's done all the time
  - 3. other data may be overwritten
  - 4. error message from the compiler
- (e) When you pass an array as an argument to a function, what actually gets passed?
  - 1. address of the array
  - 2. values of the elements of the array
  - 3. address of the first element of the array
  - 4. number of elements of the array

- (f) Which of these are reasons for using pointers?
1. To manipulate parts of an array
  2. To refer to keywords such as **for** and **if**
  3. To return more than one value from a function
  4. To refer to particular programs more conveniently
- (g) If you don't initialize a static array, what would be the elements set to?
1. 0
  2. an undetermined value
  3. a floating point number
  4. the character constant '\0'

[H] State True or False:

- (a) Address of a floating-point variable is always a whole number.
- (b) Which of the following is the correct way of declaring a float pointer:
5. `float ptr ;`
  6. `float *ptr ;`
  7. `*float ptr ;`
  8. None of the above
- (c) Add the missing statement for the following program to print 35.

```
main()
{
 int j, *ptr ;
 *ptr = 35 ;
 printf ("\n%d", j) ;
}
```



- (d) if **int s[5]** is a one-dimensional array of integers, which of the following refers to the third element in the array?

- 9.  $*(s + 2)$
- 10.  $*(s + 3)$
- 11.  $s + 3$
- 12.  $s + 2$

**[I]** Attempt the following:

- (a) Write a program to copy the contents of one array into another in the reverse order.
- (b) If an array **arr** contains **n** elements, then write a program to check if **arr[0] = arr[n-1]**, **arr[1] = arr[n-2]** and so on.
- (c) Find the smallest number in an array using pointers.
- (d) Write a program which performs the following tasks:
  - initialize an integer array of 10 elements in **main( )**
  - pass the entire array to a function **modify( )**
  - in **modify( )** multiply each element of array by 3
  - return the control to **main( )** and print the new array elements in **main( )**
- (e) The screen is divided into 25 rows and 80 columns. The characters that are displayed on the screen are stored in a special memory called VDU memory (not to be confused with ordinary memory). Each character displayed on the screen occupies two bytes in VDU memory. The first of these bytes contains the ASCII value of the character being displayed, whereas, the second byte contains the colour in which the character is displayed.

For example, the ASCII value of the character present on zeroth row and zeroth column on the screen is stored at

location number 0xB8000000. Therefore the colour of this character would be present at location number 0xB8000000 + 1. Similarly ASCII value of character in row 0, col 1 will be at location 0xB8000000 + 2, and its colour at 0xB8000000 + 3.

With this knowledge write a program which when executed would keep converting every capital letter on the screen to small case letter and every small case letter to capital letter. The procedure should stop the moment the user hits a key from the keyboard.

This is an activity of a rampant Virus called Dancing Dolls. (For monochrome adapter, use 0xB0000000 instead of 0xB8000000).

### More than one dimension

[J] What would be the output of the following programs:

```
(a) main()
{
 int n[3][3] = {
 2, 4, 3,
 6, 8, 5,
 3, 5, 1
 };
 printf ("\n%d %d %d", *n, n[3][3], n[2][2]);
}
```

```
(b) main()
{
 int n[3][3] = {
 2, 4, 3,
 6, 8, 5,
 3, 5, 1
 };
 int i, *ptr ;
```

```
ptr = n ;
for (i = 0 ; i <= 8 ; i++)
 printf ("\n%d", *(ptr + i)) ;
}

(c) main()
{
 int n[3][3] = {
 2, 4, 3,
 6, 8, 5,
 3, 5, 1
 };

 int i, j ;
 for (i = 0 ; i <= 2 ; i++)
 for (j = 0 ; j <= 2 ; j++)
 printf ("\n%d %d", n[i][j], *(n + i) + j) ;
}
```

**[K]** Point out the errors, if any, in the following programs:

```
(a) main()
{
 int twod[][] = {
 2, 4,
 6, 8
 };

 printf ("\n%d", twod) ;
}

(b) main()
{
 int three[3][] = {
 2, 4, 3,
 6, 8, 2,
 2, 3, 1
 };

 printf ("\n%d", three[1][1]) ;
}
```

```
}
```

**[L]** Attempt the following:

- (a) How will you initialize a three-dimensional array **threed[3][2][3]**? How will you refer the first and last element in this array?
- (b) Write a program to pick up the largest number from any 5 row by 5 column matrix.
- (c) Write a program to obtain transpose of a 4 x 4 matrix. The transpose of a matrix is obtained by exchanging the elements of each row with the elements of the corresponding column.
- (d) Very often in fairs we come across a puzzle that contains 15 numbered square pieces mounted on a frame. These pieces can be moved horizontally or vertically. A possible arrangement of these pieces is shown below:

|    |    |    |    |
|----|----|----|----|
| 1  | 4  | 15 | 7  |
| 8  | 10 | 2  | 11 |
| 14 | 3  | 6  | 13 |
| 12 | 9  | 5  |    |

Figure 8.12

As you can see there is a blank at bottom right corner. Implement the following procedure through a program:

Draw the boxes as shown above. Display the numbers in the above order. Allow the user to hit any of the arrow keys (up, down, left, or right).

If user hits say, right arrow key then the piece with a number 5 should move to the right and blank should replace the original position of 5. Similarly, if down arrow key is hit, then 13 should move down and blank should replace the original position of 13. If left arrow key or up arrow key is hit then no action should be taken.

The user would continue hitting the arrow keys till the numbers aren't arranged in ascending order.

Keep track of the number of moves in which the user manages to arrange the numbers in ascending order. The user who manages it in minimum number of moves is the one who wins.

How do we tackle the arrow keys? We cannot receive them using **scanf( )** function. Arrow keys are special keys which are identified by their 'scan codes'. Use the following function in your program. It would return the scan code of the arrow key being hit. Don't worry about how this function is written. We are going to deal with it later. The scan codes for the arrow keys are:

up arrow key – 72 down arrow key – 80  
left arrow key – 75 right arrow key – 77

```
/* Returns scan code of the key that has been hit */
#include "dos.h"
getkey()
{
 union REGS i, o ;
```

```

while (!kbhit())
;
i.h.ah = 0 ;
int86 (22, &i, &o);
return (o.h.ah);
}

```

- (e) Those readers who are from an Engineering/Science background may try writing programs for following problems.

- (1) Write a program to add two 6 x 6 matrices.
- (2) Write a program to multiply any two 3 x 3 matrices.
- (3) Write a program to sort all the elements of a 4 x 4 matrix.
- (4) Write a program to obtain the determinant value of a 5 x 5 matrix.

- (f) Match the following with reference to the following program segment:

```

int i, j, = 25;
int *pi, *pj = &j;
.....
..... /* more lines of program */
.....
*pj = j + 5;
j = *pj + 5 ;
pj = pj ;
*pi = i + j

```

Each integer quantity occupies 2 bytes of memory. The value assigned to **i** begin at (hexadecimal) address F9C and the value assigned to **j** begins at address F9E. Match the value represented by left hand side quantities with the right.

- |           |           |
|-----------|-----------|
| 1.    &i  | a.    30  |
| 2.    &j  | b.    F9E |
| 3.    pj  | c.    35  |
| 4.    *pj | d.    FA2 |

- |     |              |    |             |
|-----|--------------|----|-------------|
| 5.  | i            | e. | F9C         |
| 6.  | pi           | f. | 67          |
| 7.  | *pi          | g. | unspecified |
| 8.  | ( pi + 2 )   | h. | 65          |
| 9.  | (*pi + 2)    | i. | F9E         |
| 10. | * ( pi + 2 ) | j. | F9E         |
|     |              | k. | FAO         |
|     |              | l. | F9D         |

(g) Match the following with reference to the following segment:

```
int x[3][5] = {
 { 1, 2, 3, 4, 5 },
 { 6, 7, 8, 9, 10 },
 { 11, 12, 13, 14, 15 }
}, *n = &x ;
```

- |     |                      |    |    |
|-----|----------------------|----|----|
| 1.  | *( *( x + 2 ) + 1)   | a. | 9  |
| 2.  | *( *x + 2 ) + 5      | b. | 13 |
| 3.  | *( *( x + 1) )       | c. | 4  |
| 4.  | *( *( x ) + 2 ) + 1  | d. | 3  |
| 5.  | * ( *( x + 1 ) + 3 ) | e. | 2  |
| 6.  | *n                   | f. | 12 |
| 7.  | *( n + 2 )           | g. | 14 |
| 8.  | *(n + 3 ) + 1        | h. | 7  |
| 9.  | *(n + 5)+1           | i. | 1  |
| 10. | ++*n                 | j. | 8  |
|     |                      | k. | 5  |
|     |                      | l. | 10 |
|     |                      | m. | 6  |

(h) Match the following with reference to the following program segment:

```
struct
{
 int x, y;
} s[] = { 10, 20, 15, 25, 8, 75, 6, 2 };
int *i;
i = s ;
```

|     |                                                |    |     |
|-----|------------------------------------------------|----|-----|
| 1.  | <code>*( i + 3 )</code>                        | a. | 85  |
| 2.  | <code>s[i[7]].x</code>                         | b. | 2   |
| 3.  | <code>s[ (s + 2)-&gt;y / 3[I]].y</code>        | c. | 6   |
| 4.  | <code>i[i[1]-i[2]]</code>                      | d. | 7   |
| 5.  | <code>i[s[3].y]</code>                         | e. | 16  |
| 6.  | <code>( s + 1 )-&gt;x + 5</code>               | f. | 15  |
| 7.  | <code>*( 1 +i )** ( i + 4 ) / *i</code>        | g. | 25  |
| 8.  | <code>s[i[0] - i[4]].y + 10</code>             | h. | 8   |
| 9.  | <code>( *(s + *( i + 1 ) / *i ) ).x + 2</code> | i. | 1   |
| 10. | <code>++i[i[6]]</code>                         | j. | 100 |
|     |                                                | k. | 10  |
|     |                                                | l. | 20  |

- (i) Match the following with reference to the following program segment:

```
unsigned int arr[3][3] = {
 2, 4, 6,
 9, 1, 10,
 16, 64, 5
};
```

|     |                                                          |    |    |
|-----|----------------------------------------------------------|----|----|
| 1.  | <code>**arr</code>                                       | a. | 64 |
| 2.  | <code>**arr &lt; *( *arr + 2 )</code>                    | b. | 18 |
| 3.  | <code>*( arr + 2 ) / ( *( *arr + 1 ) &gt; **arr )</code> | c. | 6  |
| 4.  | <code>*( arr[1] + 1 )   arr[1][2]</code>                 | d. | 3  |
| 5.  | <code>*( arr[0] )   *( arr[2] )</code>                   | e. | 0  |
| 6.  | <code>arr[1][1] &lt; arr[0][1]</code>                    | f. | 16 |
| 7.  | <code>arr[2][[1] &amp; arr[2][0]</code>                  | g. | 1  |
| 8.  | <code>arr[2][2]   arr[0][1]</code>                       | h. | 11 |
| 9.  | <code>arr[0][1] ^ arr[0][2]</code>                       | i. | 20 |
| 10. | <code>++**arr + --arr[1][1]</code>                       | j. | 2  |
|     |                                                          | k. | 5  |
|     |                                                          | l. | 4  |

- (j) Write a program that interchanges the odd and even components of an array.
- (k) Write a program to find if a square matrix is symmetric.



- (l) Write a function to find the norm of a matrix. The norm is defined as the square root of the sum of squares of all elements in the matrix.
- (m) Given an array **p[5]**, write a function to shift it circularly left by two positions. Thus, if p[0] = 15, p[1]= 30, p[2] = 28, p[3]= 19 and p[4] = 61 then after the shift p[0] = 28, p[1] = 19, p[2] = 61, p[3] = 15 and p[4] = 30. Call this function for a (4 x 5 ) matrix and get its rows left shifted.
- (n) A 6 x 6 matrix is entered through the keyboard and stored in a 2-dimensional array **mat[7][7]**. Write a program to obtain the Determinant values of this matrix.
- (o) For the following set of sample data, compute the standard deviation and the mean.

-6, -12, 8, 13, 11, 6, 7, 2, -6, -9, -10, 11, 10, 9, 2

The formula for standard deviation is

$$\frac{\sqrt{(x_i - \bar{x})^2}}{n}$$

where  $x_i$  is the data item and  $\bar{x}$  is the mean.

- (p) The area of a triangle can be computed by the sine law when 2 sides of the triangle and the angle between them are known.

Area = (1 / 2 ) ab sin ( angle )

Given the following 6 triangular pieces of land, write a program to find their area and determine which is largest,

| Plot No. | a     | b     | angle |
|----------|-------|-------|-------|
| 1        | 137.4 | 80.9  | 0.78  |
| 2        | 155.2 | 92.62 | 0.89  |
| 3        | 149.3 | 97.93 | 1.35  |

|   |       |        |      |
|---|-------|--------|------|
| 4 | 160.0 | 100.25 | 9.00 |
| 5 | 155.6 | 68.95  | 1.25 |
| 6 | 149.7 | 120.0  | 1.75 |

- (q) For the following set of n data points (x, y), compute the correlation coefficient r, given by

$$r = \frac{\sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$$

| x     | y      |
|-------|--------|
| 34.22 | 102.43 |
| 39.87 | 100.93 |
| 41.85 | 97.43  |
| 43.23 | 97.81  |
| 40.06 | 98.32  |
| 53.29 | 98.32  |
| 53.29 | 100.07 |
| 54.14 | 97.08  |
| 49.12 | 91.59  |
| 40.71 | 94.85  |
| 55.15 | 94.65  |

- (r) For the following set of point given by (x, y) fit a straight line given by

$$y = a + bx$$

where,

$$a = \bar{y} - b\bar{x} \quad \text{and}$$

$$b = \frac{n \sum yx - \sum x \sum y}{[n \sum x^2 - (\sum x)^2]}$$

| x   | y   |
|-----|-----|
| 3.0 | 1.5 |